Exercise Sheet 08

Whole Columns at Once

House rules.

- First: `pip install -r requirements.txt` inside the lesson folder (your venv must be active).
- One file per exercise, in `my_work/`. Run from the lesson folder: `python my_work/ex1_first_array.py`, so that `data/brightwell_payroll.csv` is found.
- Exercises 2–5 and 7 start with the **loader** from Exercise 1. Copy it from your own file and extend it: collect `weekly_hours`, `gender` or `department` into extra lists as needed.
- Compare with the **expected output** character by character. Rounding matters: use `.round(2)` on arrays, `round(x, 2)` on single numbers.
- Hand in: commit and push to `lesson-08`, then open a pull request. Tell Lars which loop you were glad not to write.

Exercise 1 Your first column

10 min · `ex1_first_array.py`

> **From Priya (Payroll):** *"The salary column. All 200 of them. Lowest, highest, average, median, total. MACRO-9000 does it in a loop that takes 40 seconds and sometimes skips row 1."*

GIVEN (THE LOADER)

```
import csv
import numpy as np

salaries = []
with open("data/brightwell_payroll.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        salaries.append(float(row["annual_salary"]))

salaries = np.array(salaries)
```

(a) Print `type(salaries)`, `salaries.shape`, `salaries.dtype` and the first five values (`salaries[:5]`).

(b) Print the lowest, highest, average (rounded to 2 dp), median and total. Everything you need is a method on the array, except the median, which is `np.median(...)`.

EXPECTED OUTPUT

```
<class 'numpy.ndarray'>
(200,)
float64
[41900. 26550. 24950. 29000. 31750.]
Lowest: 10300.0
Highest: 68750.0
Average: 31999.25
Median: 29375.0
Total wage bill: 6399850.0
```

Hint: An array is a column. `(200,)` means 200 rows and no second dimension. `float64` is Excel's "Number".

Exercise 2 Dragging a formula down the column

10 min · `ex2_column_maths.py`

> **From Priya:** *"In Excel I write `=B2/12` and drag it down 200 rows. What's the Python for 'drag down'?"*

Extend the loader so it also collects `weekly_hours` into an array called `hours`. Then, **without writing a single loop**, build these arrays:

(a) `monthly` = salary / 12; `hourly` = salary / 52 / hours; `fte_salary` = salary scaled up to 37.5 hours; `with_raise` = salary + 3 %.

(b) Print the first three of each, rounded to 2 dp.

(c) Print what the 3 % raise costs in total, and the lowest and highest hourly rate.

EXPECTED OUTPUT

```
Monthly: [3491.67 2212.5  2079.17]
Hourly: [21.49 13.62 12.79]
FTE: [41900. 26550. 24950.]
After 3% raise: [43157.  27346.5 25698.5]
Raise costs: 191995.5
Lowest hourly rate: 10.26
Highest hourly rate: 35.26
```

Hint: Array divided by array divides row by row: `salaries / hours`. Array times a number multiplies every row. That's the whole trick.

Exercise 3 AutoFilter, but faster

10 min · `ex3_masks.py`

> **From Siobhan (Reward):** *"How many people earn over £40k? How many are part-time? How many part-timers in Sales? I have AutoFilter open in four windows and I've lost track."*

Collect `hours` and `department` too (`departments = np.array(departments)` works for text as well).

(a) `high = salaries > 40000`. Print `high[:5]`, how many are `True` (`.sum()`) and the average salary of those people (`salaries[high]`).

(b) `full_time = hours == 37.5`. Print how many are full-time and how many are not (`~full_time`).

(c) `sales = departments == "Sales"`. Print how many, and their average salary.

(d) Combine masks: part-time *and* Sales (`&`), Sales *or* IT (`|`), full-time *and* over £40k.

EXPECTED OUTPUT

```
[ True False False False False]
Earning over £40k: 40
Their average: 49520.0
Full time: 174
Part time: 26
Sales: 35
Sales average: 32264.29
Part-time in Sales: 6
Sales or IT: 55
Full-time over £40k: 39
```

Hint: With arrays it's `&`, `|` and `~`, not `and`, `or`, `not`, and every comparison needs its own brackets: `(a > 1) & (b == 2)`.

Exercise 4 The gap, properly this time

10 min · `ex4_gender_split.py`

> **From Gerald (MD):** *"In lesson 01 you gave me the gap for two made-up people. The board noticed. All 200, please."*

Collect `hours` and `gender`. Build `hourly`, then the masks `women` and `men`.

(a) Print how many women and men there are.

(b) Print the mean and the median hourly rate for each group, to 2 dp.

(c) Print the mean gap and the median gap as percentages of the men's figure, to 1 dp.

(d) Print how many women and how many men work fewer than 37.5 hours.

EXPECTED OUTPUT

```
Women: 83  Men: 117
Mean hourly:   women £14.95, men £18.62
Median hourly: women £13.46, men £16.56
Mean gap: 19.7%
Median gap: 18.7%
Part-time women: 10 of 83
Part-time men: 16 of 117
```

Hint: The first line is `print("Women:", women.sum(), " Men:", men.sum())`. Yes, two spaces before Men: that's what `print` does with a comma.

Exercise 5 Pay quartiles

10 min · `ex5_quartiles.py`

> **From Gerald:** *"The pay gap report has a thing called quartiles. Apparently we have to say what share of each quarter is women. Do we?"*

Collect `hours` and `gender`; build `hourly` and the `women` mask.

(a) `q1, q2, q3 = np.percentile(hourly, [25, 50, 75])`. Print the three cut-offs to 2 dp.

(b) `lower` = hourly rate at or below q1; `upper` = above q3. Print how many people are in each.

(c) Print the share of women in the lower quartile, in the upper quartile, and overall, as percentages with 1 dp.

EXPECTED OUTPUT

```
Quartile cut-offs: £13.22, £15.42, £19.86
In lower quartile: 50
In upper quartile: 50
Women in lower quartile: 72.0%
Women in upper quartile: 18.0%
Women overall: 41.5%
```

Hint: `women.mean()` on a True/False array is the share of True. For a share you'll need `(lower & women).sum() / lower.sum()`.

Exercise 6 A whole range at once

10 min · `ex6_two_dimensions.py`

> **From Mo (IT):** *"Priya's spreadsheet has people down the side and months across the top. She SUMs it both ways. Can an array do that?"*

GIVEN

```
import numpy as np

# Rows: Priya, Mo, Siobhan.  Columns: Jan..Jun monthly pay.
pay = np.array([
    [2800, 2800, 2800, 2900, 2900, 2900],
    [3100, 3100, 3100, 3100, 3300, 3300],
    [2500, 2500, 2600, 2600, 2600, 2600],
])
```

(a) Print the shape, then the total per person (`axis=1`), per month (`axis=0`), and the grand total.

(b) Print Mo's April pay (`pay[row, column]`), Siobhan's whole row, and March for everyone (`pay[:, 2]`).

(c) Print each person's best month, and a True/False array saying who got a raise in April (April > March).

EXPECTED OUTPUT

```
(3, 6)
Per person (sum along the row): [17100 19000 15400]
Per month (sum down the column): [8400 8400 8500 8600 8800 8800]
Grand total: 51500
Mo in April: 3100
Siobhan, all months: [2500 2500 2600 2600 2600 2600]
March, everyone: [2800 3100 2600]
Best month per person: [2900 3300 2600]
Raised in April: [ True False False]
```

Hint: `axis=0` squashes the rows together (you get one number per column); `axis=1` squashes the columns (one per row). Everyone gets this backwards at least once.

Exercise 7 What if? STRETCH

15 min · `ex7_what_if.py`

> **From Gerald:** *"Two ideas for the board. A: 5 % raise for everyone below the median hourly rate. B: nobody earns under £13.50 an hour. What does each do to the gap, and what does it cost?"*

Collect `hours` and `gender`; build `hourly`. Write a function `mean_gap(hourly, genders)` that returns the mean gap in percent, and use it three times.

(a) Print today's gap.

(b) Scenario A: `np.where(condition, value_if_true, value_if_false)` is `IF()` dragged down a column. Build the new hourly array, print its gap, and the yearly cost: the hourly increase × hours × 52, summed.

(c) Scenario B: `np.maximum(hourly, 13.50)`. Print its gap, its cost, and how many people it affects.

EXPECTED OUTPUT

```
Gap today: 19.7%
Scenario A gap: 18.3%
Scenario A cost per year: £120,887.50
Scenario B gap: 17.1%
```

```
Scenario B cost per year: £124,145.00
People affected by B: 61
```

Bug hunt Dave's Leftovers #8

15 min · `bughunt/quartile_bug.py`

Dave found NumPy in his last month and wrote a "quartile report" for the board. It crashed, and he emailed Gerald the error message by mistake. **Three** bugs: one crashes with an error message you haven't seen before (read all of it), one gives a wrong number without complaining (compare with your Exercise 5), one gets a sign wrong.

EXPECTED OUTPUT ONCE FIXED

```
Upper quartile starts at: £19.86
Full-time earning over £20/h: 44
Mean gap: 19.7%
```

One commit per bug, each message explaining what was wrong and why.